

UNIVERSIDAD DE SANTIAGO DE CHILE

FACULTAD DE INGENIERÍA

Departamento de Ingeniería Informática



**Informe Nro. 2: Capacitated Vehicle Routing
Problem (CVRP) mediante *Particle Swarm
Optimization* (PSO) y comparación con el
método exacto**

Cristopher René Alexander Angulo Ahumada

Profesora: Dra. Mónica Villanueva I.

Profesora: Dr. Mario Inostroza P.

Curso: Optimización en Ingeniería

Programa: Magíster en Ingeniería Informática

Santiago – Chile

2026

RESUMEN

El *Capacitated Vehicle Routing Problem* (CVRP) es un problema clásico de optimización combinatoria en el que se debe determinar un conjunto de rutas de costo mínimo para abastecer clientes con demandas conocidas mediante una flota homogénea de vehículos con capacidad limitada. En el Informe Nro. 1 este problema se abordó mediante métodos exactos de programación matemática, cuyo esfuerzo computacional crece de forma abrupta con el tamaño de la instancia. El presente informe aborda el CVRP desde la perspectiva de las *metaheurísticas*, implementando *Particle Swarm Optimization* (PSO) como método aproximado capaz de obtener soluciones de alta calidad sin certificado de optimalidad, y contrasta ambos enfoques sobre un conjunto común de instancias. Para adaptar PSO, un algoritmo de naturaleza continua, al espacio combinatorio del CVRP, se implementó la representación SR-1 (Ai & Kachitvichyanukul, 2009), que decodifica cada partícula a una lista de prioridad de clientes y puntos de referencia de vehículos, construyendo rutas factibles mediante inserción de menor costo y mejora local 2-opt. Los hiperparámetros de PSO se calibraron con Optuna (60 *trials*) sobre instancias de tuning distintas de las de evaluación. El método se evaluó con 31 repeticiones independientes sobre nueve instancias de CVRPLIB: las cinco del Informe 1 (para comparación directa) y cuatro adicionales de mayor tamaño, fuera del alcance del método exacto. En las cinco instancias compartidas, PSO obtuvo un GAP promedio entre 0,03 % y 10,07 % respecto al óptimo, en tiempos de ejecución de fracciones de segundo, frente a un método exacto que llegó a tardar 4,7 horas sin certificar optimalidad en la instancia más grande compartida. En las cuatro instancias adicionales, el GAP promedio se ubicó entre 15,25 % y 26,84 %, con el máximo en la instancia más grande. La comparación evidencia una complementariedad entre ambos enfoques: el método exacto certifica optimalidad mientras es tratable, pero pierde esa tratabilidad de forma abrupta más allá de cierto tamaño; PSO no certifica, pero mantiene tiempos acotados y resuelve instancias considerablemente mayores que las alcanzables por el método exacto.

Palabras clave: CVRP; optimización combinatoria; metaheurísticas; *Particle Swarm Optimization*; PSO; CVRPLIB.

ABSTRACT

The *Capacitated Vehicle Routing Problem* (CVRP) is a classical combinatorial optimization problem in which a minimum-cost set of routes must be designed to serve customers with known demands by means of a homogeneous fleet of capacity-constrained vehicles. In Report No. 1 the problem was addressed through exact mathematical programming methods, whose computational effort grows sharply with instance size. This report tackles the CVRP from the standpoint of *metaheuristics*, implementing *Particle Swarm Optimization* (PSO) as an approximate method able to obtain high-quality solutions without a certificate of optimality, and contrasts both approaches over a common set of instances. To adapt PSO, a continuous-domain algorithm, to the combinatorial search space of the CVRP, the SR-1 representation (Ai & Kachitvichyanukul, 2009) was implemented, which decodes each particle into a customer priority list and vehicle reference points, building feasible routes through least-cost insertion and 2-opt local improvement. PSO hyperparameters were calibrated with Optuna (60 trials) on tuning instances distinct from the evaluation set. The method was evaluated with 31 independent repetitions over nine CVRPLIB instances: the five from Report 1 (for direct comparison) and four additional, larger instances beyond the reach of the exact method. On the five shared instances, PSO achieved an average GAP between 0,03 % and 10,07 % with respect to the optimum, in sub-second runtimes, versus an exact method that took up to 4,7 hours without certifying optimality on the largest shared instance. On the four additional instances, the average GAP ranged from 15,25 % to 26,84 %, with the maximum on the largest instance. The comparison reveals a complementarity between both approaches: the exact method certifies optimality while it remains tractable, but loses tractability abruptly beyond a certain size; PSO does not certify optimality, but keeps bounded runtimes and solves instances considerably larger than those reachable by the exact method.

Keywords: CVRP; combinatorial optimization; metaheuristics; *Particle Swarm Optimization*; PSO; CVRPLIB.

Índice

1	INTRODUCCIÓN	1
1.1	Contexto y motivación.....	1
1.2	Fundamentos teóricos.....	2
1.3	Objetivos.....	3
1.4	Organización del informe	4
2	DEFINICIÓN Y FORMULACIÓN DEL PROBLEMA	5
2.1	Definición del problema.....	5
2.2	Formulación matemática	5
3	ESTADO DEL ARTE	7
3.1	Metaheurísticas para el CVRP	7
3.2	Particle Swarm Optimization	8
3.3	Adaptación de PSO a problemas combinatorios y al CVRP	8
3.3.1	Codificación de valor real y decodificación.....	8
3.3.2	PSO discreto	9
3.3.3	Enfoques híbridos.....	9
3.4	Posicionamiento del enfoque adoptado.....	9
4	MÉTODO	11
4.1	Adquisición de datos e instancias.....	11
4.2	Diseño del algoritmo PSO	12
4.2.1	Representación: codificación y decodificación.....	12
4.2.2	Función de aptitud	14
4.2.3	Dinámica del enjambre	14
4.2.4	Pseudocódigo.....	15
4.2.5	Parámetros del algoritmo.....	16
4.3	Parametrización	17
4.4	Implementación computacional.....	18

4.5	Diseño experimental y protocolo de comparación	19
5	RESULTADOS Y ANÁLISIS	22
5.1	Condiciones experimentales.....	22
5.2	Resultados del método PSO	22
5.3	Análisis de convergencia	24
5.4	Comparación con el método exacto	25
6	DISCUSIÓN	27
6.1	Exacto vs. metaheurística.....	27
6.2	Limitaciones y trabajo futuro	28
7	CONCLUSIONES	30
A	Notación principal	32
B	Instancias de referencia	33
C	Código implementado	33
	REFERENCIAS BIBLIOGRÁFICAS	35

ÍNDICE DE FIGURAS

Figura 1:	Diseño experimental: las instancias de tuning (parametrización) están separadas de las de evaluación (experimentación), y el método exacto del Informe 1 se incorpora directamente en la etapa de análisis, sin volver a ejecutarse. Fuente: elaboración propia.....	20
Figura 2:	Distribución del GAP % de PSO por instancia (31 corridas). En rojo, las cuatro instancias fuera del alcance del método exacto. Fuente: elaboración propia.....	23
Figura 3:	Curvas de convergencia de PSO (GAP % vs. iteración) para una instancia chica (A-n32-k5) y una grande (E-n101-k8). Línea: mediana de 31 corridas; banda: rango mejor-peor. Fuente: elaboración propia.	24
Figura 4:	Convergencia de PSO en tiempo real (escala log) para las cinco instancias compartidas con el método exacto. El punto rojo marca el resultado final del método exacto (estrella: óptimo certificado; cruz: no certificado, truncado por el límite de 60 s por resolución del modelo entero). Fuente: elaboración propia.	26

CAPÍTULO 1: INTRODUCCIÓN

Los problemas de ruteo de vehículos ocupan un lugar central en la investigación de operaciones, ya que articulan decisiones de asignación, secuenciamiento y transporte bajo restricciones operacionales. Entre sus variantes, el *Capacitated Vehicle Routing Problem* (CVRP) destaca como un modelo base para estudiar la distribución física de mercancías desde un depósito central hacia clientes con demanda conocida, cuando la capacidad de carga de cada vehículo es limitada (Toth & Vigo, 2002).

En el Informe Nro. 1 este problema se abordó mediante métodos exactos de programación matemática, capaces de certificar optimalidad sobre instancias pequeñas y medianas, pero cuyo esfuerzo computacional crece de forma abrupta con el tamaño de la instancia. El presente informe complementa ese trabajo abordando el CVRP mediante una *metaheurística*, *Particle Swarm Optimization* (PSO), y contrastando ambos enfoques sobre un conjunto común de instancias, según lo requerido por el enunciado del trabajo de investigación.

1.1 Contexto y motivación

El interés práctico por el CVRP proviene de la necesidad de diseñar redes de distribución con bajo costo total y alto nivel de servicio. En operaciones logísticas reales, una planificación de rutas deficiente incrementa kilómetros recorridos, horas-hombre, consumo de combustible y uso de flota, deteriorando la competitividad del sistema. Desde una perspectiva de modelación, el CVRP representa la interacción entre decisiones de partición de clientes en rutas y decisiones de ordenamiento de visitas dentro de cada recorrido, lo que lo convierte en un problema estructuralmente complejo.

Dos contextos de aplicación ilustran esta relevancia. En distribución de retail y comercio mayorista, un centro de distribución debe abastecer múltiples locales o puntos de venta con una flota propia o subcontratada; aquí, el cumplimiento de capacidad y la consolidación eficiente de carga son determinantes para reducir costos de transporte y frecuencia de despachos. En recolección de residuos domiciliarios, los camiones deben visitar sectores urbanos completos, respetando la capacidad de compactación antes de

retornar al depósito o estación de descarga; en este caso, la factibilidad operacional de cada ruta depende directamente de la demanda acumulada por sector. En ambos escenarios, el CVRP entrega una representación matemática suficientemente rica para apoyar decisiones tácticas y operacionales.

1.2 Fundamentos teóricos

El CVRP pertenece a la familia de problemas $\mathcal{NP}$ -hard y generaliza, entre otros, al problema del viajante de comercio cuando la capacidad es suficientemente grande o cuando se utiliza un solo vehículo (Lenstra & Rinnooy Kan, 1981). Esta dificultad combinatoria motiva, más allá de los métodos exactos, el uso de *metaheurísticas*: procedimientos de búsqueda de alto nivel que exploran el espacio de soluciones guiados por reglas heurísticas, sacrificando la garantía de optimalidad a cambio de obtener soluciones de buena calidad en tiempos acotados sobre instancias que quedan fuera del alcance de los métodos exactos.

Las metaheurísticas se dividen, a grandes rasgos, en aquellas de *trayectoria* (refinan una única solución candidata, como la búsqueda tabú o el temple simulado) y aquellas *basadas en población*, que evolucionan un conjunto de soluciones candidatas en paralelo. *Particle Swarm Optimization* (PSO), introducido por Kennedy y Eberhart (1995), pertenece a esta segunda familia: un *enjambre* de partículas explora el espacio de soluciones imitando el comportamiento colectivo de organismos sociales (bandadas de aves, cardúmenes). Cada partícula i mantiene una posición $\mathbf{x}_i$, que codifica una solución candidata, y una velocidad $\mathbf{v}_i$, que representa su desplazamiento en cada iteración; la velocidad se actualiza combinando tres influencias: la inercia de su movimiento anterior, la atracción hacia su mejor posición histórica ($pbest$) y la atracción hacia la mejor posición encontrada por el enjambre ($gbest$), ponderadas respectivamente por un peso de inercia y por coeficientes de aceleración cognitivo y social (Poli et al., 2007).

PSO fue formulado originalmente para dominios *continuos*: la posición de una partícula es un vector real y su actualización es una simple suma vectorial posición-velocidad. El CVRP, en cambio, es un problema *combinatorio*: una solución es un conjunto de rutas, no un punto en $\mathbb{R}^d$, y no existe una noción directa de “sumar una velocidad” a una partición de clientes en rutas. Este desajuste de dominios es el desafío

central que debe resolverse para aplicar PSO al CVRP, y exige definir un esquema de *codificación* (cómo un vector real representa una solución del CVRP) y su correspondiente procedimiento de *decodificación* (cómo recuperar un conjunto de rutas factible a partir de ese vector). La decisión de diseño concreta adoptada en este trabajo para resolver ese desajuste se presenta en el Capítulo 4, tras revisar en el Capítulo 3 las alternativas consideradas en la literatura.

1.3 Objetivos

El objetivo general de este informe es resolver el CVRP mediante la metaheurística *Particle Swarm Optimization* y comparar su desempeño con el método exacto implementado en el Informe Nro. 1, sobre instancias estándar de la literatura.

Los objetivos específicos son los siguientes:

1. Revisar el estado del arte de las metaheurísticas aplicadas al CVRP, posicionando PSO dentro de dicho panorama.
2. Definir un esquema de representación (codificación y decodificación) que permita aplicar PSO, algoritmo de naturaleza continua, sobre el espacio de soluciones combinatorio del CVRP.
3. Implementar computacionalmente el algoritmo PSO para el CVRP, incluyendo el manejo de las restricciones de capacidad y la construcción de rutas factibles.
4. Evaluar el método sobre instancias de los Sets E y A de CVRPLIB (las mismas empleadas en el Informe 1) y, adicionalmente, sobre instancias de mayor tamaño que exceden el alcance del método exacto.
5. Contrastar los resultados de PSO frente al método exacto en términos de calidad de solución (gap respecto del óptimo o de la mejor solución conocida), tiempo de ejecución y escalabilidad.

1.4 Organización del informe

El Capítulo 2 presenta la definición del problema y su formulación matemática sobre un grafo no dirigido. El Capítulo 3 resume el estado del arte de las metaheurísticas para el CVRP, posicionando PSO dentro de ese panorama. El Capítulo 4 describe el diseño del algoritmo PSO, con énfasis en el esquema de codificación/decodificación, la parametrización y el protocolo experimental. El Capítulo 5 reporta los resultados de la experimentación computacional. El Capítulo 6 discute los hallazgos y desarrolla la comparación entre el enfoque exacto y el metaheurístico. Finalmente, el Capítulo 7 presenta las conclusiones.

CAPÍTULO 2: DEFINICIÓN Y FORMULACIÓN DEL PROBLEMA

2.1 Definición del problema

El *Capacitated Vehicle Routing Problem* (CVRP) consiste en diseñar un conjunto de rutas de costo mínimo para abastecer a un conjunto de clientes con demanda conocida, empleando una flota homogénea de vehículos con capacidad limitada que parten y retornan a un depósito central. Cada cliente debe ser visitado exactamente una vez por un único vehículo, y la demanda acumulada de los clientes asignados a una ruta no puede exceder la capacidad del vehículo.

2.2 Formulación matemática

Sea $G = (V, E)$ un grafo completo no dirigido, donde $V = \{1, 2, \dots, n\}$ es el conjunto de nodos, con el nodo 1 representando el depósito, y E el conjunto de aristas. Cada cliente $i \in V \setminus \{1\}$ posee una demanda $d_i \geq 0$, y cada arista $\{i, j\} \in E$ tiene un costo $c_{ij} = c_{ji}$ (problema simétrico). Se considera una flota homogénea de m vehículos, todos con capacidad D . Una solución es un conjunto de rutas $\mathcal{R}$, cada una un ciclo que parte y retorna al depósito, tal que cada cliente es visitado exactamente una vez y la demanda acumulada de cada ruta no excede D .

La función objetivo minimiza la distancia total recorrida por la flota:

$$\min z = \sum_{r \in \mathcal{R}} \sum_{\{i,j\} \in r} c_{ij}, \quad (1)$$

sujeta a las condiciones de factibilidad: cada cliente pertenece exactamente a una ruta (las rutas particionan $V \setminus \{1\}$) y cada ruta $r \in \mathcal{R}$ respeta la capacidad,

$$\sum_{i \in r} d_i \leq D, \quad \forall r \in \mathcal{R}. \quad (2)$$

Esta es la misma definición del CVRP abordada en el Informe 1. La diferencia radica en el método de resolución: mientras el enfoque exacto formulaba el problema como un *problema lineal entero* y lo resolvía sobre un grafo no dirigido, en este trabajo la

metaheurística PSO no resuelve el modelo entero de forma directa, sino que explora el espacio de soluciones evaluando (1) como función de aptitud y garantizando (2) en la etapa de decodificación (Capítulo 4).

CAPÍTULO 3: ESTADO DEL ARTE

El CVRP ha sido objeto de estudio sistemático en la literatura de investigación operacional desde su introducción a fines de los años cincuenta (Dantzig & Ramser, 1959). Su carácter $\mathcal{NP}$ -hard (Lenstra & Rinnooy Kan, 1981) explica que, junto a los métodos exactos revisados en el Informe 1, se haya desarrollado un amplio cuerpo de *métodos aproximados* capaces de escalar a instancias industriales. En este informe el alcance se centra en las metaheurísticas y, en particular, en *Particle Swarm Optimization*.

3.1 Metaheurísticas para el CVRP

Las metaheurísticas suelen clasificarse en métodos basados en *trayectoria*, que operan sobre una única solución que se modifica iterativamente (como *Tabu Search* o *Simulated Annealing*), y métodos basados en *población*, que mantienen y hacen evolucionar un conjunto de soluciones, como los algoritmos genéticos, *Ant Colony Optimization* (ACO) y *Particle Swarm Optimization* (PSO). La literatura de metaheurísticas para el CVRP es extensa y abarca desde algoritmos constructivos con búsqueda local hasta esquemas poblacionales sofisticados. Entre los métodos de trayectoria destacan la búsqueda tabú y el recocido simulado; entre los poblacionales, los algoritmos genéticos, ACO y PSO; y en años recientes, los esquemas de búsqueda de vecindario amplio (LNS/ALNS) figuran entre los enfoques aproximados más competitivos para las variantes del VRP (Tan & Yeh, 2021). Todos estos métodos renuncian a la garantía de optimalidad a cambio de escalar a instancias de tamaño industrial, y con frecuencia se combinan con búsqueda local para intensificar la exploración del espacio de soluciones.

El presente trabajo se sitúa dentro de los métodos poblacionales, adoptando PSO. Una revisión específica de su aplicación al problema de ruteo, con una comparación de sus principales variantes, se encuentra en Marinakis et al. (2018), que sirve de referencia para los apartados siguientes.

3.2 Particle Swarm Optimization

PSO fue introducido por Kennedy y Eberhart (1995) como un algoritmo de optimización inspirado en el comportamiento social de bandadas de aves y cardúmenes de peces. Cada *partícula* representa una solución candidata que se desplaza por el espacio de búsqueda con una *velocidad* actualizada en función de su mejor posición histórica (*pbest*) y de la mejor posición del enjambre o del vecindario (*gbest*). Contribuciones posteriores refinaron su dinámica: el *peso de inercia* (Shi & Eberhart, 1998) y el *factor de constricción* (Clerc & Kennedy, 2002) regulan el equilibrio entre exploración y explotación y garantizan estabilidad de la convergencia. Una síntesis unificada de estas variantes se encuentra en Poli et al. (2007).

3.3 Adaptación de PSO a problemas combinatorios y al CVRP

La formulación canónica de PSO opera sobre un espacio continuo $\mathbb{R}^d$, mientras que el CVRP es un problema combinatorio cuyas soluciones son permutaciones y particiones de clientes. Como las nociones de velocidad y dirección carecen de un significado natural sobre recorridos y permutaciones (Poli et al., 2007), aplicar PSO al CVRP exige un mecanismo que relacione las posiciones continuas del enjambre con soluciones de ruteo factibles. La literatura ha abordado esta brecha por tres vías principales.

3.3.1 Codificación de valor real y decodificación

La vía más difundida conserva la dinámica continua de PSO y delega la traducción al espacio combinatorio a un procedimiento de *decodificación*: la posición continua de una partícula se interpreta, típicamente ordenando sus componentes, como una lista de prioridad o una permutación de clientes, que luego se particiona en rutas respetando la capacidad. Ai y Kachitvichyanukul (2009) propusieron dos representaciones de referencia para el CVRP. La primera, SR-1, es un vector de $n + 2m$ dimensiones cuyas primeras n componentes inducen una lista de prioridad de clientes (por orden ascendente de su valor) y cuyas últimas $2m$ componentes fijan puntos de referencia que orientan la asignación de clientes a vehículos; la segunda, SR-2, extiende esta idea con una representación alternativa. Ambas incorporan procedimientos de mejora local en la decodificación para elevar la

calidad de las soluciones. Sobre esta misma línea, Ahmed y Sun (2018) introdujeron un método de decodificación novedoso combinado con una búsqueda local en dos capas (BLS-PSO), que alcanza las mejores soluciones conocidas en la mayoría de las instancias de prueba; su resultado ilustra el papel determinante que cumplen tanto la decodificación como la hibridación con búsqueda local en el desempeño final.

3.3.2 PSO discreto

Una segunda vía redefine los operadores de PSO directamente sobre el espacio discreto, reinterpretando la velocidad como una secuencia de intercambios (*swaps*) que transforma una permutación en otra. Chen et al. (2006) desarrollaron un PSO discreto (DPSO) para el CVRP que combina búsqueda global y local y se hibrida con recocido simulado para escapar de óptimos locales, reportando buen desempeño en instancias de gran tamaño. En una línea afín, Mauliddina et al. (2020) implementaron un DPSO para el CVRP y ajustaron sus parámetros mediante un diseño experimental de un factor a la vez (OFAT), contrastando el resultado con modelos de PSO propuestos previamente.

3.3.3 Enfoques híbridos

Una tercera vía combina PSO con otras metaheurísticas o con esquemas de agrupamiento. Kao et al. (2012) propusieron un híbrido de ACO y PSO para el CVRP en el que cada hormiga memoriza su mejor solución, al estilo de una partícula. Son y Tan (2021) adoptaron un enfoque en dos fases (agrupamiento de clientes mediante k -Means seguido de una fase de ruteo con un híbrido adaptativo de PSO y *Grey Wolf Optimization*), obteniendo soluciones cercanas a las de los métodos exactos. Estas variantes evidencian una tendencia general: el PSO puro rara vez alcanza por sí solo las mejores soluciones conocidas, por lo que la hibridación con búsqueda local u otros mecanismos de intensificación es habitual en los mejores resultados reportados para el CVRP.

3.4 Posicionamiento del enfoque adoptado

El Informe 1 mostró que el esquema exacto certifica optimalidad sobre instancias pequeñas y medianas, pero pierde tratabilidad al crecer el tamaño del problema: el salto de

$n = 32$ a $n = 33$ bastó para desplazar la instancia fuera del rango resoluble a optimalidad dentro del presupuesto de tiempo. Este límite motiva el uso de una metaheurística, capaz de obtener soluciones de buena calidad en tiempos acotados sobre instancias que exceden el alcance del método exacto. Se adopta PSO por tratarse de un método poblacional ampliamente estudiado, de implementación relativamente simple y con una dinámica de búsqueda bien caracterizada (Poli et al., 2007).

De las tres vías de adaptación revisadas en la Sección 3.3, este trabajo adopta la codificación de valor real con decodificación, siguiendo la representación SR de Ai y Kachitvichyanukul (2009). La elección responde a tres razones. Primero, preserva la dinámica continua de PSO, de modo que las ecuaciones canónicas de velocidad y posición se aplican sin modificación, a diferencia del PSO discreto (Chen et al., 2006), que exige redefinir la velocidad como secuencias de intercambios. Segundo, constituye una representación de referencia para el CVRP, con un procedimiento de decodificación documentado que garantiza la factibilidad de capacidad de forma constructiva. Tercero, evita la complejidad de los esquemas híbridos o en dos fases (Kao et al., 2012; Son & Tan, 2021), cuya infraestructura excede el alcance de este trabajo y dificultaría aislar el comportamiento propio de PSO.

La revisión también muestra que los mejores resultados, como la obtención de las mejores soluciones conocidas, suelen provenir de variantes híbridadas con búsqueda local (Ahmed & Sun, 2018). No obstante, el objetivo de este informe no es competir por la mejor solución conocida, sino caracterizar el compromiso entre el método exacto y una metaheurística representativa; por ello se opta por una implementación de PSO con representación SR, transparente y bien comprendida, cuyo detalle se desarrolla en el Capítulo 4. La incorporación de búsqueda local se contempla como una posible extensión.

CAPÍTULO 4: MÉTODO

4.1 Adquisición de datos e instancias

La evaluación del método se apoya en instancias estándar de la biblioteca *Capacitated Vehicle Routing Problem Library* (CVRPLIB), repositorio ampliamente utilizado para contrastar algoritmos sobre un conjunto común de casos de prueba y soluciones de referencia (CVRPLIB, 2024). Para permitir una comparación directa con el método exacto del Informe 1, se reutilizan las mismas cinco instancias de los Sets E y A; adicionalmente, se incorporan cuatro instancias de mayor tamaño que quedaron fuera del alcance del enfoque exacto, con el fin de evaluar la escalabilidad de PSO.

Cada archivo .vrp se procesa para extraer el número de nodos, la capacidad vehicular D , y la demanda de cada nodo. La mayoría de las instancias sigue la convención EUC_2D de TSPLIB utilizada por CVRPLIB, en la que el costo de cada arista es la distancia euclidiana entre coordenadas, redondeada al entero más cercano, $c_{ij} = \left\lceil \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \right\rceil$. La instancia E-n13-k4 constituye una excepción: usa el tipo EXPLICIT (formato LOWER_ROW), en el que la matriz de distancias se entrega directamente y no existen coordenadas. Dado que la representación SR-1 (Sección 4.2.1) requiere ubicar puntos de referencia de vehículos en un plano 2D, para esta instancia se generó un embedding aproximado mediante *escalamiento multidimensional clásico* (*classical MDS*) a partir de la matriz de distancias real; esta reconstrucción solo se usa como insumo interno del mecanismo de referencia del decodificador, mientras que todos los costos reportados se calculan siempre con la matriz de distancias original de la instancia, nunca con distancias derivadas del embedding.

La Tabla 1 detalla el conjunto final de instancias evaluadas, distinguiendo aquellas con óptimo certificado (por el método exacto del Informe 1) de aquellas para las que solo se dispone de la mejor solución conocida (BKS, *best known solution*), que no es necesariamente óptima.

Cuadro 1: Instancias de evaluación. n incluye el depósito; m es el número de vehículos de la solución de referencia.

Instancia	n	m	Referencia	Tipo de referencia
E-n13-k4	13	4	247	Óptimo certificado (Informe 1)
E-n22-k4	22	4	375	Óptimo certificado (Informe 1)
E-n23-k3	23	3	569	Óptimo certificado (Informe 1)
A-n32-k5	32	5	784	Óptimo certificado (Informe 1)
A-n33-k5	33	5	661	BKS (Informe 1 no certificó: 667 tras 4,7 h)
A-n60-k9	60	9	1354	BKS (CVRPLIB)
A-n80-k10	80	10	1763	BKS (CVRPLIB)
E-n76-k10	76	10	830	BKS (CVRPLIB)
E-n101-k8	101	8	815	BKS (CVRPLIB)

Las cuatro instancias bajo la línea separadora quedaron fuera del alcance del método exacto: extrapolando el crecimiento observado en el Informe 1 (576,1 s en $n = 32$ a 17 096,4 s, 4,7 h, en $n = 33$, sin certificar), resolverlas exactamente habría requerido un tiempo de cómputo no disponible para este trabajo.

4.2 Diseño del algoritmo PSO

4.2.1 Representación: codificación y decodificación

El aspecto central del diseño es la traducción entre el espacio continuo en que opera PSO y el espacio combinatorio de soluciones del CVRP. Se adoptó la representación **SR-1** de Ai y Kachitvichyanukul (2009), que mantiene la formulación canónica de PSO sin modificar sus ecuaciones de velocidad/posición (Sección 4.2.1 del Capítulo 3 argumenta esta elección frente a las alternativas de PSO discreto e híbridos).

Codificación. Para una instancia con $n - 1$ clientes y m vehículos, una partícula es un vector real de $(n - 1) + 2m$ dimensiones. Las primeras $n - 1$ componentes son claves de prioridad, una por cliente; las siguientes $2m$ componentes fijan m puntos de referencia

$(x_j^{\text{ref}}, y_j^{\text{ref}})$, uno por vehículo, en el mismo plano donde están los clientes.

Decodificación. A partir de la partícula se construyen, en orden:

1. **Lista de prioridad de clientes.** Se ordenan las primeras $n - 1$ componentes de menor a mayor valor (*Smallest Position Value*, SPV); el orden resultante de los índices de cliente es la lista de prioridad.
2. **Matriz de prioridad de vehículos.** Para cada cliente, se ordenan los m vehículos de menor a mayor distancia entre el cliente y el punto de referencia de cada vehículo.
3. **Construcción de rutas.** Se recorre la lista de prioridad de clientes; a cada cliente se lo intenta insertar, en la posición de menor costo adicional, en la ruta del vehículo de mayor prioridad para ese cliente que aún tenga capacidad disponible; si ningún vehículo lo admite sin violar la capacidad, se prueba con el siguiente en la prioridad del cliente. Tras cada inserción exitosa se aplica de inmediato el procedimiento de mejora local 2-opt sobre esa ruta.

Ejemplo numérico. Se reproduce el ejemplo original de Ai y Kachitvichyanukul (2009) para 6 clientes y 2 vehículos. Sea la partícula

$$\mathbf{x} = (0,4, 0,2, 0,9, 0,5, 0,6, 0,1 \mid 0,8, 1,5, 0,3, 1,2),$$

donde las primeras 6 componentes son las claves de los clientes $1, \dots, 6$ y las últimas 4 codifican los puntos de referencia $(0,8; 0,3)$ del Vehículo 1 y $(1,5; 1,2)$ del Vehículo 2. Ordenando las claves de los clientes de menor a mayor $(0,1 < 0,2 < 0,4 < 0,5 < 0,6 < 0,9)$ se obtiene la lista de prioridad 6, 2, 1, 4, 5, 3. Calculando la distancia de cada cliente a ambos puntos de referencia se obtiene, por ejemplo, que los clientes 1 y 2 están más cerca del Vehículo 2, mientras que los clientes 3, 4, 5 y 6 están más cerca del Vehículo 1. Insertando en ese orden de prioridad, respetando la capacidad, y aplicando 2-opt, se obtienen las rutas Vehículo 1: 0-6-3-4-5-0 y Vehículo 2: 0-2-1-0.

Factibilidad. A diferencia de esquemas basados en un *split* sobre una gran gira (que abren una nueva ruta cada vez que se excede la capacidad), en SR-1 el número de vehículos m es fijo y viene dado por la partícula misma. Esto implica que, en casos raros, ningún vehículo puede aceptar a un cliente sin violar su capacidad: el propio Ai y Kachitvichyanukul (2009) reporta este fenómeno (Tabla 2 de ese trabajo, instancia *vrpnc9*: “*all replications yielded infeasible solution*”). La Sección 4.2.2 describe cómo se trata este caso en la implementación.

4.2.2 Función de aptitud

La aptitud de una partícula es el costo total (1) de las rutas obtenidas al decodificarla. Cuando la decodificación deja uno o más clientes sin asignar a ningún vehículo (Sección 4.2.1), se añade una penalización fija y grande por cada cliente no asignado, de modo que estas partículas queden efectivamente descartadas por PSO sin impedir su evaluación. No se penaliza el exceso de capacidad propiamente tal, porque el mecanismo de inserción de SR-1 nunca genera una ruta que exceda D : por construcción, un cliente solo se inserta en un vehículo si su capacidad lo permite; el único modo de falla es que *ningún* vehículo lo admita, no que se viole la capacidad de alguno.

4.2.3 Dinámica del enjambre

Cada partícula i mantiene una posición $\mathbf{x}_i$ y una velocidad $\mathbf{v}_i$ en $\mathbb{R}^d$. En cada iteración, la velocidad y la posición se actualizan según la formulación canónica con peso de inercia:

$$\mathbf{v}_i(t+1) = w \mathbf{v}_i(t) + c_1 \mathbf{r}_1 \otimes (\mathbf{p}_i - \mathbf{x}_i(t)) + c_2 \mathbf{r}_2 \otimes (\mathbf{p}_g - \mathbf{x}_i(t)), \quad (3)$$

$$\mathbf{x}_i(t+1) = \mathbf{x}_i(t) + \mathbf{v}_i(t+1), \quad (4)$$

donde $\mathbf{p}_i$ es la mejor posición histórica de la partícula (*pbest*), $\mathbf{p}_g$ la mejor posición global del enjambre (*gbest*, topología global, sin subdivisión en vecindarios), w el peso de inercia, c_1 y c_2 los coeficientes cognitivo y social, $\mathbf{r}_1, \mathbf{r}_2 \sim U(0, 1)^d$ vectores aleatorios y $\otimes$ el producto de Hadamard (componente a componente). El peso de inercia decae linealmente

de w_0 a w_f a lo largo de la corrida. La velocidad se acota componente a componente a $[-v_{\text{máx}}, v_{\text{máx}}]$; la posición se acota al rango $[0, 1]$ (rango de las claves SPV y de los puntos de referencia), reflejando la velocidad a 0 en las componentes que alcanzan el borde. El criterio de término es un número fijo de iteraciones T (Sección 4.3).

4.2.4 Pseudocódigo

El Algoritmo 1 resume el esquema general de PSO para el CVRP, integrando la codificación, la decodificación a rutas, la evaluación de aptitud y la actualización del enjambre. La mejora local (2-opt) no se aplica como un paso adicional separado al final de cada iteración, sino que ya queda incorporada dentro de la propia decodificación (Sección 4.2.1), inmediatamente después de insertar cada cliente.

Algoritmo 1 PSO para el CVRP (representación SR-1)

- 1: **Procedimiento** PSO-CVRP(instancia, N , w_0 , w_f , c_1 , c_2 , $v_{\text{máx}}$, T)
 - 2: Inicializar posiciones $\mathbf{x}_i \sim U(0, 1)^d$ y velocidades $\mathbf{v}_i \leftarrow \mathbf{0}$ del enjambre de N partículas
 - 3: **Para** cada partícula i **hacer**
 - 4: Decodificar $\mathbf{x}_i$ a rutas y evaluar aptitud (Sección 4.2.2)
 - 5: $\mathbf{p}_i \leftarrow \mathbf{x}_i$
 - 6: **fin Para**
 - 7: $\mathbf{p}_g \leftarrow \arg \min_i \text{aptitud}(\mathbf{p}_i)$
 - 8: **Para** $t \leftarrow 1 \dots T$ **hacer**
 - 9: $w \leftarrow w_0 + \frac{t}{T}(w_f - w_0)$
 - 10: **Para** cada partícula i **hacer**
 - 11: Actualizar $\mathbf{v}_i$ (Ec. 3) y $\mathbf{x}_i$ (Ec. 4); acotar a $v_{\text{máx}}$ y a $[0, 1]$
 - 12: Decodificar $\mathbf{x}_i$ y evaluar aptitud
 - 13: **Si** aptitud mejora a $\mathbf{p}_i$ **entonces** $\mathbf{p}_i \leftarrow \mathbf{x}_i$
 - 14: **Si** aptitud mejora a $\mathbf{p}_g$ **entonces** $\mathbf{p}_g \leftarrow \mathbf{x}_i$
 - 15: **fin Para**
 - 16: **fin Para**
 - 17: **Devolver** rutas decodificadas de $\mathbf{p}_g$
 - 18: **fin Procedimiento**
-

4.2.5 Parámetros del algoritmo

Los parámetros del algoritmo y sus valores se resumen en la Tabla 2. Su calibración se aborda en la Sección 4.3.

Cuadro 2: Parámetros del algoritmo PSO, encontrados mediante parametrización con Optuna (Sección 4.3).

Parámetro	Símbolo	Valor
Tamaño del enjambre	N	100
Peso de inercia inicial	w_0	0,9116
Peso de inercia final	w_f	0,2782
Coefficiente cognitivo	c_1	2,4045
Coefficiente social	c_2	1,1843
Velocidad máxima	$v_{\text{máx}}$	0,3762 (fracción del rango $[0, 1]$)
Criterio de parada	T	500 iteraciones (derivado del presupuesto, ver más abajo)
Repeticiones por instancia	—	31

4.3 Parametrización

El desempeño de PSO depende de sus parámetros (N , w_0 , w_f , c_1 , c_2 , $v_{\text{máx}}$), por lo que su calibración se realiza de forma sistemática y reproducible, separando la fase de *parametrización* de la fase de *evaluación* para evitar sesgo por sobreajuste: el ajuste se efectúa sobre un subconjunto de instancias de *tuning* distinto del conjunto de instancias sobre el que luego se reportan los resultados (Tabla 1).

Herramienta. Se usó Optuna (Akiba et al., 2019), con su muestreador bayesiano por defecto (*Tree-structured Parzen Estimator*, TPE). Se prefirió Optuna sobre *irace* porque el flujo de trabajo completo (parametrización, experimentación, análisis) se mantiene íntegramente en Python, sin depender de R como herramienta externa.

Presupuesto de trials. Se ejecutaron 60 *trials*, siguiendo la heurística de fijar un presupuesto de aproximadamente $10 \times$ el número de hiperparámetros a tunear (6 en este caso).

Espacio de búsqueda e instancias de tuning. La Tabla 3 resume el espacio de búsqueda explorado. Se usaron tres instancias de tuning de tamaños crecientes y ninguna

coincide con las de evaluación: A-n34-k5 ($n = 34$, chica), A-n62-k8 ($n = 62$, mediana) y E-n76-k7 ($n = 76$, grande). Se usa más de una instancia, y de tamaños distintos, para que los parámetros encontrados generalicen razonablemente entre escalas, y no solo a la escala de una única instancia pequeña.

Cuadro 3: Espacio de búsqueda explorado por Optuna.

Parámetro	Rango/valores
N	$\{20, 30, 50, 80, 100\}$
w_0	$[0,5, 0,95]$
w_f	$[0,1, 0,5]$
c_1	$[0,5, 2,5]$
c_2	$[0,5, 2,5]$
$v_{\text{máx}}$ (fracción del rango)	$[0,1, 1,0]$

Presupuesto de evaluaciones y derivación de T . Cada corrida de PSO dispone de un presupuesto fijo de 50 000 evaluaciones de la función objetivo, igual en la parametrización y en la experimentación final. Como N es parte del espacio de búsqueda, el número de iteraciones se deriva de él, $T = \lfloor 50\,000/N \rfloor$, de modo que todas las configuraciones evaluadas por Optuna consumen el mismo esfuerzo de cómputo independientemente de cuántas partículas usen.

Resultado. La mejor configuración encontrada (Tabla 2) obtuvo un GAP promedio de 8,54 % sobre las tres instancias de tuning (3,34 % en A-n34-k5, 7,92 % en A-n62-k8 y 14,37 % en E-n76-k7). El GAP crece con el tamaño de la instancia de tuning, un patrón que se retoma en la Sección 5.4 al discutir cómo generaliza esta parametrización a las instancias de evaluación más grandes.

4.4 Implementación computacional

La implementación se desarrolló en Python 3.12, apoyándose en NumPy (Harris et al., 2020) para las operaciones vectorizadas del enjambre y en Numba (Lam et al., 2015)

(compilación *just-in-time*, modo `nopython`) para acelerar el núcleo de decodificación SR-1 (construcción de rutas y 2-opt), que opera con estructuras de largo variable difíciles de vectorizar directamente con NumPy. Esta aceleración fue necesaria en la práctica: la versión en Python puro evaluaba entre 680 y 2 850 partículas por segundo según el tamaño de la instancia, lo que habría hecho impracticable la parametrización completa con Optuna (más de 2 horas solo para esa fase); la versión acelerada con Numba, validada para dar resultados idénticos a la versión de referencia en Python puro, evalúa entre 130 000 y 200 000 partículas por segundo, reduciendo la parametrización completa (60 *trials*) a menos de un minuto. La búsqueda de hiperparámetros se realizó con Optuna 4.9 (Akiba et al., 2019).

Los experimentos se ejecutaron localmente en un equipo de escritorio con procesador Intel Core i9-11900K (8 núcleos, 3,50 GHz), 32 GB de RAM y Windows 11 Home. Este es el mismo equipo, con la misma configuración de hardware, en el que se ejecutó la experimentación definitiva del método exacto del Informe 1: si bien las pruebas iniciales de ese informe se validaron en Google Colab, los tiempos reportados y utilizados en la comparación de la Sección 5.4 (incluidos los 576,1 s de A-n32-k5 y los 17 096,4 s de A-n33-k5) provienen de la ejecución definitiva en este mismo equipo local. La comparación de tiempos entre ambos métodos no está, por lo tanto, sujeta a un sesgo de infraestructura.

4.5 Diseño experimental y protocolo de comparación

La Figura 1 resume el flujo experimental completo, desde la parametrización hasta la comparación final con el método exacto.

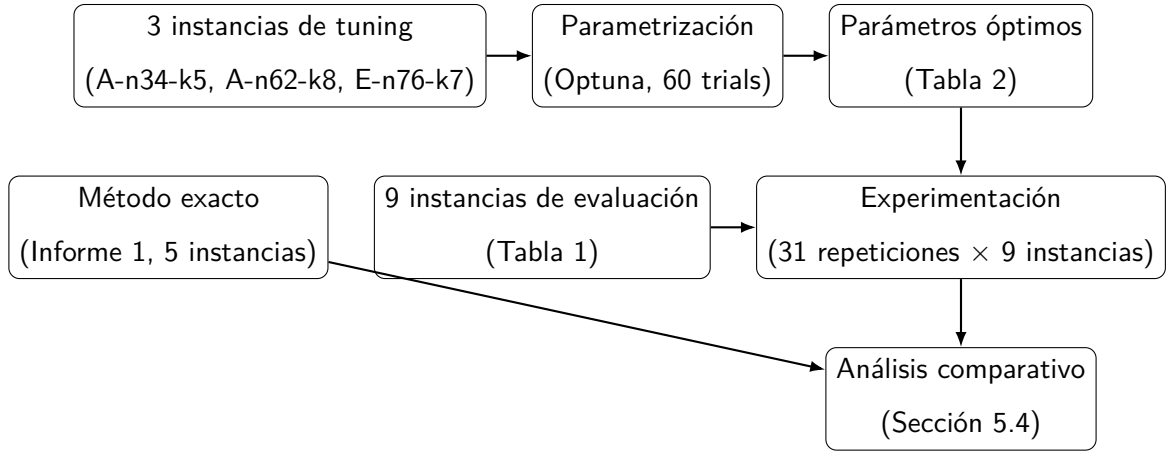


Figura 1: Diseño experimental: las instancias de tuning (parametrización) están separadas de las de evaluación (experimentación), y el método exacto del Informe 1 se incorpora directamente en la etapa de análisis, sin volver a ejecutarse. Fuente: elaboración propia.

La **variable independiente** evaluada es el método de resolución (exacto vs. PSO), contrastado por instancia; dentro de PSO, la semilla de cada repetición es una fuente de variabilidad estocástica controlada mediante las 31 repeticiones independientes, no una variable de interés en sí misma. A diferencia del método exacto, que es determinista, PSO es un algoritmo *estocástico*: distintas semillas producen distintas trayectorias de búsqueda. Por ello, cada instancia se resuelve con 31 repeticiones independientes (semillas 0, ..., 30), y se reportan estadísticos agregados (mejor, promedio, desviación estándar) en lugar de un único valor. No se aplica un test estadístico inferencial en la comparación central de este informe porque uno de los dos métodos comparados (el exacto) es determinista y entrega un único valor por instancia, no una distribución: la comparación pertinente es directa, verificando dónde se ubica ese valor único respecto de la distribución completa de las 31 corridas de PSO, que se reporta íntegramente (Figura 2). Los tests no paramétricos de comparación de muestras (como Wilcoxon o Friedman) aplican al comparar dos o más algoritmos estocásticos entre sí, caso que no se da aquí.

Las métricas registradas son: el costo de la mejor solución encontrada y el costo promedio $\pm$ desviación estándar sobre las 31 repeticiones; el GAP porcentual respecto al óptimo o BKS de cada instancia,

$$\text{GAP} (\%) = \frac{z_{\text{obtenido}} - z_{\text{referencia}}}{z_{\text{referencia}}} \times 100; \quad (5)$$

y el tiempo de ejecución promedio por corrida.

El contraste entre el método exacto y PSO (requisito 8 del enunciado, Sección 5.4) no iguala el presupuesto de cómputo entre ambos métodos: mientras que igualar el número de evaluaciones de la función objetivo tendría sentido al comparar dos metaheurísticas entre sí, no es aplicable aquí, porque el método exacto no tiene una noción de “evaluaciones” comparable y su costo temporal es una consecuencia directa de certificar (o no) la optimalidad, no un presupuesto que se le imponga externamente. Por eso, contra el método exacto se reporta el costo propio de cada enfoque tal como se ejecutó en su respectivo informe.

CAPÍTULO 5: RESULTADOS Y ANÁLISIS

5.1 Condiciones experimentales

Los resultados que siguen se obtuvieron con la configuración de PSO de la Tabla 2 (Sección 4.3), sobre las 9 instancias de la Tabla 1, con 31 repeticiones independientes por instancia (semillas $0, \dots, 30$) y un presupuesto de 50 000 evaluaciones de la función objetivo por corrida. Se registran, por instancia: el costo de la mejor solución encontrada, el costo promedio y su desviación estándar sobre las 31 corridas, el GAP % respecto al óptimo o BKS de la Tabla 1, y el tiempo de ejecución promedio por corrida. El entorno de ejecución se describe en la Sección 4.3 (equipo personal, Python/NumPy/Numba/Optuna).

5.2 Resultados del método PSO

La Tabla 4 resume el desempeño de PSO sobre las 9 instancias evaluadas.

Cuadro 4: Resultados de PSO sobre instancias de CVRPLIB. Estadísticos sobre 31 repeticiones independientes por instancia. Ref. es el óptimo certificado o BKS de la Tabla 1; las últimas cuatro filas (bajo la línea) corresponden a instancias fuera del alcance del método exacto.

Instancia	n	Ref.	Mejor	Prom.	Desv.	Gap prom. (%)	Tiempo (s)
E-n13-k4	13	247	251	256,6	3,0	3,89	0,066
E-n22-k4	22	375	376	392,5	12,3	4,66	0,086
E-n23-k3	23	569	569	569,2	0,4	0,03	0,096
A-n32-k5	32	784	786	816,7	25,4	4,17	0,106
A-n33-k5	33	661	697	727,6	16,2	10,07	0,106
A-n60-k9	60	1354	1486	1560,5	56,3	15,25	0,172
E-n76-k10	76	830	924	1013,2	40,3	22,07	0,212
A-n80-k10	80	1763	1911	2045,4	71,3	16,02	0,226
E-n101-k8	101	815	954	1033,7	44,2	26,84	0,306

En cuatro de las cinco instancias compartidas con el método exacto ($n \leq 33$),

PSO se mantiene por debajo del 5 % de GAP promedio, e incluso alcanza el óptimo exacto en E-n23-k3; la excepción es A-n33-k5 (10,07 %). En las cuatro instancias adicionales ($n \geq 60$), el GAP promedio se mueve entre 15,25 % y 26,84 %, con una tendencia creciente respecto del tamaño que no es estrictamente monótona: E-n76-k10 (22,07 %) supera a A-n80-k10 (16,02 %) pese a ser más chica, lo que indica que la dificultad no depende solo de n sino también de la estructura de cada instancia (la razón demanda/capacidad y el número de vehículos difieren entre ambas familias). La Figura 2 (construida a partir de los mismos datos de la tabla) muestra este patrón mediante la distribución completa del GAP % de las 31 corridas por instancia, no solo su promedio.

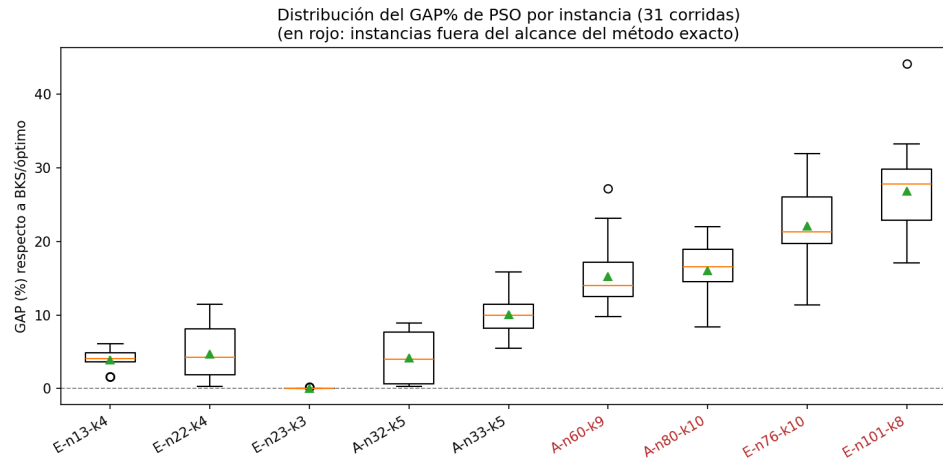


Figura 2: Distribución del GAP % de PSO por instancia (31 corridas). En rojo, las cuatro instancias fuera del alcance del método exacto. Fuente: elaboración propia.

La tendencia creciente del GAP con el tamaño es consistente con lo observado ya en la parametrización (Sección 4.3): las instancias de tuning más grandes (A-n62-k8, GAP 7,92 %; E-n76-k7, GAP 14,37 %) también mostraron GAP mayor que la chica (A-n34-k5, GAP 3,34 %). Además, las dos instancias de evaluación más grandes ($n = 80$ y $n = 101$) exceden el tamaño de la mayor instancia de tuning usada ($n = 76$), por lo que en ese rango los parámetros operan fuera de la escala en que fueron calibrados; se retoma esta observación en la discusión.

5.3 Análisis de convergencia

La Figura 3 muestra la evolución del GAP % del mejor global del enjambre en función de la iteración, para una instancia chica compartida con el método exacto (A-n32-k5, $n = 32$) y la instancia más grande del conjunto de evaluación (E-n101-k8, $n = 101$). La línea es la corriente mediana de las 31 repeticiones; la banda sombreada es el rango entre la mejor y la peor corrida en cada iteración.

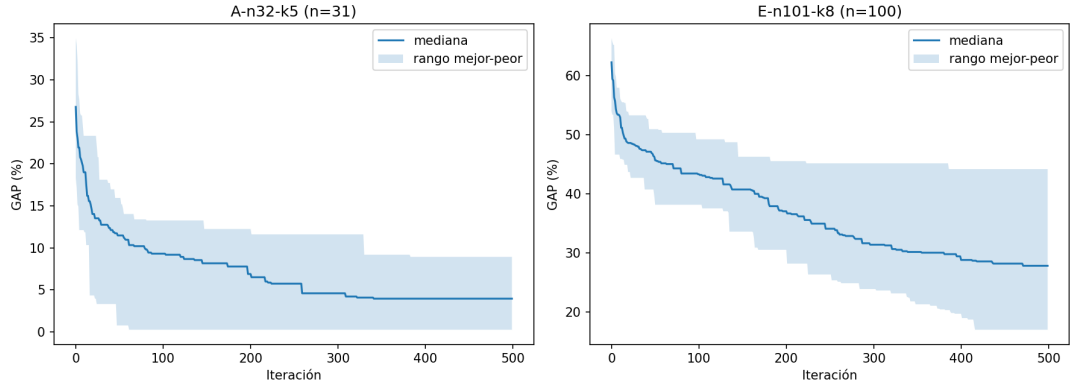


Figura 3: Curvas de convergencia de PSO (GAP % vs. iteración) para una instancia chica (A-n32-k5) y una grande (E-n101-k8). Línea: mediana de 31 corridas; banda: rango mejor-peor. Fuente: elaboración propia.

En A-n32-k5, la última mejora de la curva mediana ocurre en la iteración 341 de las 500 disponibles (Tabla 2): el último tercio de la corrida transcurre sin cambios en la mediana, es decir, el enjambre agota su margen de mejora antes de agotar su presupuesto de iteraciones. En E-n101-k8, en cambio, la mediana sigue mejorando hasta prácticamente el final (última mejora en la iteración 471 de 500), y su descenso aún es visible en el tramo final de la curva: el mismo presupuesto que sobra en la instancia chica apenas alcanza en la más grande. Dado que T se deriva de un presupuesto fijo de evaluaciones igual para todas las instancias ($T = \lfloor 50\,000/N \rfloor$, Sección 4.3), las instancias de mayor dimensión de búsqueda ($n + 2m$ crece con n) disponen relativamente de menos iteraciones para explorar un espacio más grande, lo que contribuye al mayor GAP observado en la Tabla 4 para $n \geq 60$, más allá de la calidad de la parametrización misma.

5.4 Comparación con el método exacto

La Tabla 5 contrasta el método exacto del Informe 1 con PSO sobre las cinco instancias comunes. Esta comparación constituye el objetivo central del informe (requisito 8 del enunciado).

Cuadro 5: Comparación método exacto (Informe 1) vs. PSO, sobre las cinco instancias compartidas. Gap de PSO respecto al óptimo o BKS de la Tabla 1 (661 en A-n33-k5, no 667). Tiempo de PSO: promedio de 31 corridas.

Instancia	Exacto (Informe 1)			PSO (mejor de 31)		
	z^*	Gap (%)	Tiempo (s)	Mejor	Gap (%)	Tiempo (s)
E-n13-k4	247	0,00	0,9	251	1,62	0,066
E-n22-k4	375	0,00	0,8	376	0,27	0,086
E-n23-k3	569	0,00	1,2	569	0,00	0,096
A-n32-k5	784	0,00	576,1	786	0,26	0,106
A-n33-k5	667	0,91	17 096,4	697	5,45	0,106

En las instancias más chicas ($n \leq 23$), ambos métodos son prácticamente instantáneos (menos de 1,2 s el exacto, menos de 0,1 s PSO) y PSO queda a menos de 2% del óptimo. A partir de $n = 32$ la brecha de tiempo se abre por completo: el exacto pasa de 1,2 s en E-n23-k3 a 576,1 s en A-n32-k5 y a 17 096,4 s (4,7 h) en A-n33-k5, sin siquiera certificar optimalidad en esta última (Tabla 1); PSO, en cambio, se mantiene en 0,1 s en ambas, sin variación apreciable. La razón de tiempos entre PSO y el exacto es de aproximadamente $5\,460\times$ en A-n32-k5 y $161\,000\times$ en A-n33-k5: una diferencia de varios órdenes de magnitud, medida en el mismo equipo para ambos métodos (Sección 4.3), por lo que no es atribuible a ninguna diferencia de infraestructura.

La Figura 4 presenta esta misma comparación de forma gráfica: la curva de convergencia de PSO en función del tiempo real (no de iteraciones), en escala logarítmica, junto con el resultado final del método exacto representado como un único punto, ya que este último no dispone de una curva de convergencia comparable en esta comparación (no se re-ejecutó para registrar su incumbente por iteración).

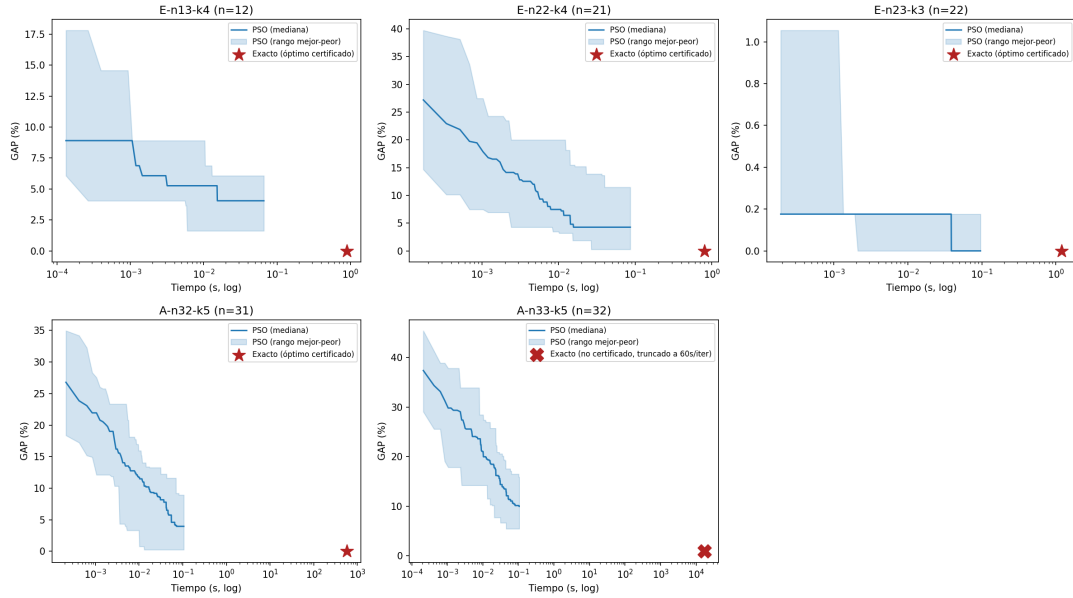


Figura 4: Convergencia de PSO en tiempo real (escala log) para las cinco instancias compartidas con el método exacto. El punto rojo marca el resultado final del método exacto (estrella: óptimo certificado; cruz: no certificado, truncado por el límite de 60 s por resolución del modelo entero). Fuente: elaboración propia.

El punto queda siempre muy a la derecha del tramo donde la curva de PSO ya se aplanó, ilustrando visualmente el mismo hallazgo de la Tabla 5: PSO alcanza su mejor solución en una fracción de segundo, mientras el exacto necesita órdenes de magnitud más tiempo para llegar a un punto (certificado o no) que, en el peor caso observado (A-n33-k5), PSO iguala o incluso mejora en calidad relativa apenas 5,45 % por encima del óptimo real, en una fracción del tiempo.

CAPÍTULO 6: DISCUSIÓN

PSO logró soluciones de calidad competitiva en las instancias chicas y medianas: en las cinco compartidas con el método exacto ($n \leq 33$), el GAP promedio se mantuvo entre 0,03 % y 10,07 %, tocando el óptimo exacto en E-n23-k3 y quedando a menos de 5 % de él en tres de las otras cuatro. Este desempeño queda, con todo, por debajo del reportado por Ai y Kachitvichyanukul (2009) para la misma representación: su SR-1, aplicada con el algoritmo GLNPSO y un conjunto más amplio de mejoras locales, alcanza el óptimo exacto de A-n33-k5 (661, Tabla 1 de ese trabajo), mientras que la mejor de las 31 corridas de este informe llegó a 697 en esa instancia; la diferencia es atribuible a la variante de PSO más simple usada aquí (Sección 6.2). En las cuatro instancias agregadas para evaluar escalabilidad ($n \geq 60$, fuera del alcance del método exacto), en cambio, el GAP promedio se ubicó entre 15,25 % y 26,84 %, con el máximo en la instancia más grande (E-n101-k8, $n = 101$): PSO sigue entregando una solución factible y razonable en todos los casos, pero su ventaja relativa en calidad se reduce claramente al alejarse del rango de tamaños usado para parametrizarlo.

6.1 Exacto vs. metaheurística

La comparación de la Sección 5.4 deja ver con claridad el compromiso entre *garantía* y *escalabilidad* que motiva este informe. El método exacto certifica optimalidad y, cuando puede hacerlo en tiempo razonable (instancias con $n \leq 32$), es la opción preferible sin ambigüedad: no hay ninguna razón para aceptar una solución aproximada cuando el óptimo se obtiene en menos de 10 minutos. Pero el salto observado en el Informe 1 entre $n = 32$ (576,1 s) y $n = 33$ (17 096,4 s, sin certificar) muestra que esa tratabilidad no se degrada suavemente: se pierde de forma abrupta con apenas un cliente adicional. PSO, en el mismo rango, no muestra ningún quiebre comparable: su tiempo de ejecución crece de forma acotada con n (de 0,066 s en $n = 13$ a 0,306 s en $n = 101$, Tabla 4), y sigue entregando una solución completa incluso en instancias que el método exacto nunca llegó a intentar.

El punto en que la metaheurística se vuelve preferible no es, entonces, un umbral

fijo de n , sino la pregunta de si la aplicación puede tolerar una solución sin garantía de optimalidad a cambio de una respuesta en tiempo acotado. Para instancias pequeñas donde el exacto es rápido, no hay motivo para preferir PSO; para instancias donde el exacto se vuelve intratable (como A-n33-k5, y con mayor razón las cuatro instancias más grandes de este informe, nunca intentadas por el exacto), PSO es la única alternativa de las dos que entrega una respuesta en un tiempo práctico, aun cuando esa respuesta pueda estar a un GAP de dos dígitos del óptimo real.

6.2 Limitaciones y trabajo futuro

La implementación tiene dos limitaciones identificables a partir de los propios resultados. Ninguna se relaciona con la parametrización en sí, que se realizó de forma completa y sistemática con Optuna, sobre un espacio de búsqueda y un presupuesto de evaluaciones explícitos (Sección 4.3); ambas provienen de decisiones de diseño del algoritmo y del protocolo experimental. Primero, se usó PSO canónico (gbest global, dos términos de velocidad, Ecuación 3), no la variante GLNPSO de Ai y Kachitvichyanukul (2009), que agrega términos de mejor local (*lbest*) y de mejor vecino según razón aptitud-distancia (*nbest*); esta simplificación probablemente explica en parte por qué el GAP obtenido aquí es algo mayor al reportado por los autores originales para instancias de tamaño comparable. Segundo, el número de iteraciones T se deriva de un presupuesto fijo de evaluaciones igual para todas las instancias ($T = \lfloor 50\,000/N \rfloor$, Sección 4.3), sin ajustarse a que la dimensión del espacio de búsqueda ($n + 2m$) crece con el tamaño de la instancia; el análisis de convergencia (Sección 5.4) mostró que la curva mediana de E-n101-k8 todavía descendía en la última iteración disponible, a diferencia de A-n32-k5, que ya se había estabilizado bastante antes de agotar su presupuesto de iteraciones. A esto se suma que las instancias de tuning utilizadas (hasta $n = 76$, Sección 4.3) no cubren todo el rango evaluado (hasta $n = 101$); esto no es una falla del proceso de parametrización, que se ejecutó igual para todas las escalas, sino una propiedad esperable de cualquier calibración empírica sobre un conjunto finito de instancias: los parámetros encontrados generalizan mejor cerca del rango en que fueron ajustados. Ambos factores (el presupuesto de iteraciones fijo y el alcance del tuning) son consistentes con el mayor

GAP observado en las instancias más grandes, y ayudan a explicarlo (Tabla 4).

Una limitación potencial de la propia representación SR-1, señalada por Ai y Kachitvichyanukul (2009) (Tabla 2 de ese trabajo, instancia *vrpnc9*), es que el mecanismo de asignación de clientes a un número fijo de vehículos puede, en casos raros, dejar clientes sin asignar. En las 279 corridas realizadas para este informe ($31 \text{ repeticiones} \times 9 \text{ instancias}$) esto no ocurrió en ningún caso, por lo que no fue una limitación práctica aquí, pero se documenta como un riesgo inherente al esquema de codificación, no eliminado por la implementación.

De estas limitaciones se desprenden mejoras concretas para trabajo futuro: incorporar los términos de vecindario social de GLNPSO (*lbest/nbest*) para acercar el desempeño al reportado en la literatura de referencia; escalar el número de iteraciones con la dimensión del problema en vez de mantenerlo fijo por presupuesto de evaluaciones; ampliar el conjunto de instancias de tuning para cubrir el rango completo de tamaños evaluados; e hibridar la búsqueda con procedimientos de mejora local adicionales más allá del 2-opt ya embebido en la decodificación (p. ej. *or-opt* o intercambios entre rutas), en la línea de lo discutido para PSO memético en el Capítulo 3.

CAPÍTULO 7: CONCLUSIONES

Este trabajo abordó el *Capacitated Vehicle Routing Problem*, problema $\mathcal{NP}$ -hard de optimización combinatoria que consiste en diseñar rutas de costo mínimo para una flota homogénea de vehículos con capacidad limitada, esta vez mediante la metaheurística *Particle Swarm Optimization*, y en contraste con el método exacto implementado en el Informe 1.

Sobre las nueve instancias evaluadas, PSO con representación SR-1 (Ai & Kachitvichyanukul, 2009) logró soluciones de calidad competitiva en el rango de tamaños compartido con el método exacto ($n \leq 33$): GAP promedio entre 0,03 % y 10,07 %, llegando al óptimo exacto en E-n23-k3, y con un tiempo de ejecución acotado (máximo 0,306 s en la instancia más grande evaluada, $n = 101$) que nunca superó el segundo, frente a un método exacto cuyo tiempo pasó de 576,1 s ($n = 32$) a 17 096,4 s ($n = 33$, sin certificar optimalidad). En las cuatro instancias adicionales fuera del alcance del método exacto (n entre 60 y 101), PSO siguió entregando soluciones factibles y completas, aunque con un GAP mayor (entre 15,25 % y 26,84 %, con el máximo en la instancia más grande), reflejo de que la parametrización y el presupuesto fijo de iteraciones no escalan explícitamente con el tamaño del problema (Sección 6.2).

El resultado central de la comparación exacto vs. metaheurística es la *complementariedad*, no la sustitución de un método por otro: el método exacto certifica optimalidad y es preferible sin ambigüedad mientras siga siendo tratable en tiempo razonable, pero esa tratabilidad se pierde de forma abrupta, no gradual: bastó un cliente adicional ($n = 32 \rightarrow 33$ en el Informe 1) para pasar de minutos a horas sin certificado. PSO no ofrece esa garantía, pero mantiene un tiempo de ejecución acotado y predecible incluso en instancias que el método exacto nunca pudo abordar, a costa de un GAP que crece con el tamaño y que, en el peor caso observado aquí, no superó el 27 %.

La implicación general es que la elección entre ambos enfoques no depende de cuál es “mejor” en abstracto, sino del tamaño de la instancia a resolver y de si la aplicación exige una garantía formal de optimalidad o puede operar con una solución de buena calidad obtenida en tiempo acotado: el método exacto certifica pero no escala; la

metaheurística escala pero no certifica.

APÉNDICE A: NOTACIÓN PRINCIPAL

Símbolo	Descripción
n	Número total de nodos del grafo, incluyendo el depósito.
V	Conjunto de nodos del grafo, $V = \{1, 2, \dots, n\}$.
$V \setminus \{1\}$	Conjunto de clientes (nodos distintos del depósito).
E	Conjunto de aristas del grafo completo no dirigido.
c_{ij}	Costo de recorrer la arista $\{i, j\}$.
d_i	Demanda del cliente i .
D	Capacidad homogénea de cada vehículo.
m	Número de vehículos.
$\mathcal{R}$	Conjunto de rutas de una solución.
z	Costo total (distancia recorrida) de una solución.
<i>Particle Swarm Optimization</i>	
N	Número de partículas del enjambre.
d	Dimensión del espacio de búsqueda de PSO.
$\mathbf{x}_i$	Posición de la partícula i (solución candidata codificada).
$\mathbf{v}_i$	Velocidad (desplazamiento) de la partícula i .
$\mathbf{p}_i$	Mejor posición histórica de la partícula i ($pbest$).
$\mathbf{p}_g$	Mejor posición del enjambre o del vecindario ($gbest$).
w	Peso de inercia.
c_1, c_2	Coeficientes cognitivo y social.
$\mathbf{r}_1, \mathbf{r}_2$	Vectores aleatorios $\sim U(0, 1)^d$.
$\otimes$	Producto de Hadamard (componente a componente).
χ	Factor de constricción.

APÉNDICE B: INSTANCIAS DE REFERENCIA

Las familias A, B y E de CVRPLIB se identifican mediante nombres del tipo A-n32-k5, donde la letra indica la familia, n corresponde al número total de nodos incluyendo depósito y k al número de vehículos de la solución de referencia. Esta convención facilita la trazabilidad experimental y la comparación entre estudios.

APÉNDICE C: CÓDIGO IMPLEMENTADO

El código completo (módulo compartido `cvrp_common.py` y los tres notebooks de parametrización, experimentación y análisis) está disponible en: [TODO: enlace al repositorio].

A continuación se incluyen dos fragmentos representativos de la implementación de referencia en Python puro (la versión efectivamente usada en las corridas está acelerada con Numba, Sección 4.3, pero es funcionalmente idéntica y fue validada contra esta).

```
def decode_sr1(x, instance, n_vehicles, apply_local_search=True):
    n = instance.n_customers
    m = n_vehicles
    dist = instance.dist
    demands = instance.demands

    priority_list = _priority_list(x[:n]) # SPV: argsort ascendente + 1
    vehicle_priority = _vehicle_priority_matrix(x[n:n + 2*m], instance.coords, m)

    routes = [[] for _ in range(m)]
    loads = np.zeros(m)
    unassigned = []

    for customer in priority_list:
        demand = demands[customer]
        placed = False
        for j in vehicle_priority[customer - 1]:
            if loads[j] + demand > instance.capacity:
                continue
            pos, _delta = _best_insertion_cost(routes[j], customer, dist)
            routes[j].insert(pos, customer)
            loads[j] += demand
            if apply_local_search:
                routes[j] = _two_opt(routes[j], dist)
```

```

        placed = True
        break
    if not placed:
        unassigned.append(customer) # penalizado en evaluate_particle
    return routes, unassigned

```

Listing 1: Decodificación SR-1: lista de prioridad de clientes (SPV) e inserción de menor costo.

```

w = params.w0 + (t / max(params.n_iter - 1, 1)) * (params.wf - params.w0)
r1 = rng.uniform(0, 1, size=(N, d))
r2 = rng.uniform(0, 1, size=(N, d))
V = w * V + params.c1 * r1 * (P - X) + params.c2 * r2 * (g[None, :] - X)
V = np.clip(V, -v_max, v_max)
X = X + V
below, above = X < x_min, X > x_max
X = np.clip(X, x_min, x_max)
V[below | above] = 0.0 # reflejo de velocidad a 0 en el borde

```

Listing 2: Actualización de velocidad y posición del enjambre (Ecuaciones 3-4).

REFERENCIAS BIBLIOGRÁFICAS

- Ahmed, A. K. M. F., & Sun, J. U. (2018). Bilayer Local Search Enhanced Particle Swarm Optimization for the Capacitated Vehicle Routing Problem. *Algorithms*, 11(3), 31. <https://doi.org/10.3390/a11030031>
- Ai, T. J., & Kachitvichyanukul, V. (2009). Particle Swarm Optimization and Two Solution Representations for Solving the Capacitated Vehicle Routing Problem. *Computers & Industrial Engineering*, 56(1), 380-387. <https://doi.org/10.1016/j.cie.2008.06.012>
- Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A Next-generation Hyperparameter Optimization Framework. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2623-2631. <https://doi.org/10.1145/3292500.3330701>
- Chen, A.-l., Yang, G.-k., & Wu, Z.-m. (2006). Hybrid Discrete Particle Swarm Optimization Algorithm for Capacitated Vehicle Routing Problem. *Journal of Zhejiang University SCIENCE A*, 7(4), 607-614. <https://doi.org/10.1631/jzus.2006.A0607>
- Clerc, M., & Kennedy, J. (2002). The Particle Swarm — Explosion, Stability, and Convergence in a Multidimensional Complex Space. *IEEE Transactions on Evolutionary Computation*, 6(1), 58-73. <https://doi.org/10.1109/4235.985692>
- CVRPLIB. (2024). CVRPLIB: Capacitated Vehicle Routing Problem Library [Repositorio de instancias benchmark y mejores soluciones conocidas para el CVRP]. <http://vrp.atd-lab.inf.puc-rio.br/>
- Dantzig, G. B., & Ramser, J. H. (1959). The Truck Dispatching Problem. *Management Science*, 6(1), 80-91. <https://doi.org/10.1287/mnsc.6.1.80>
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357-362. <https://doi.org/10.1038/s41586-020-2649-2>

- Kao, Y., Chen, M.-H., & Huang, Y.-T. (2012). A Hybrid Algorithm Based on ACO and PSO for Capacitated Vehicle Routing Problems. *Mathematical Problems in Engineering*, 2012, 726564. <https://doi.org/10.1155/2012/726564>
- Kennedy, J., & Eberhart, R. (1995). Particle Swarm Optimization. *Proceedings of the IEEE International Conference on Neural Networks (ICNN'95)*, 4, 1942-1948. <https://doi.org/10.1109/ICNN.1995.488968>
- Lam, S. K., Pitrou, A., & Seibert, S. (2015). Numba: A LLVM-based Python JIT compiler. *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, 1-6. <https://doi.org/10.1145/2833157.2833162>
- Lenstra, J. K., & Rinnooy Kan, A. H. G. (1981). Complexity of Vehicle Routing and Scheduling Problems. *Networks*, 11(2), 221-227. <https://doi.org/10.1002/net.3230110211>
- Marinakis, Y., Marinaki, M., & Migdalas, A. (2018). Particle Swarm Optimization for the Vehicle Routing Problem: A Survey and a Comparative Analysis. En R. Martí, P. M. Pardalos & M. G. C. Resende (Eds.), *Handbook of Heuristics* (pp. 1163-1189). Springer. https://doi.org/10.1007/978-3-319-07124-4_42
- Mauliddina, A. N., Saifuddin, F. A., Nagari, A. L., Redi, A. A. N. P., Kurniawan, A. C., & Ruswandi, N. (2020). Implementation of Discrete Particle Swarm Optimization Algorithm in the Capacitated Vehicle Routing Problem. *Jurnal Sistem dan Manajemen Industri*, 4(2), 117-128. <https://doi.org/10.30656/jsmi.v4i2.2607>
- Poli, R., Kennedy, J., & Blackwell, T. (2007). Particle Swarm Optimization: An Overview. *Swarm Intelligence*, 1(1), 33-57. <https://doi.org/10.1007/s11721-007-0002-0>
- Shi, Y., & Eberhart, R. (1998). A Modified Particle Swarm Optimizer. *Proceedings of the IEEE International Conference on Evolutionary Computation (IEEE World Congress on Computational Intelligence)*, 69-73. <https://doi.org/10.1109/ICEC.1998.699146>
- Son, D. V. T., & Tan, P. N. (2021). Capacitated Vehicle Routing Problem — A New Clustering Approach Based on Hybridization of Adaptive Particle Swarm Optimization and Grey Wolf Optimization. En I. Aljarah, H. Faris & S. Mirjalili

- (Eds.), *Evolutionary Data Clustering: Algorithms and Applications*. Springer. https://doi.org/10.1007/978-981-33-4191-3_5
- Tan, S.-Y., & Yeh, W.-C. (2021). The Vehicle Routing Problem: State-of-the-Art Classification and Review. *Applied Sciences*, 11(21), 10295. <https://doi.org/10.3390/app112110295>
- Toth, P., & Vigo, D. (Eds.). (2002). *The Vehicle Routing Problem*. Society for Industrial and Applied Mathematics. <https://doi.org/10.1137/1.9780898718515>